Notation Conventions

This book is chock full of code examples. To show you that a line of code results in a value, I print that value as a comment on the same line:

```
-1.abs          # => 1
```

If a code example is supposed to print a result rather than return it, I show that result after the code:

```
puts 'Testing... testing...'
```

```
Testing... testing...
```

In most cases, the text uses the same code syntax that Ruby uses: `MyClass.my_method` is a class method, `MyClass::MY_CONSTANT` is a constant defined within a class, and so on. There are a couple of exceptions to this rule. First, I identify instance methods with the *hash* notation, like the Ruby documentation does (`MyClass#my_method`). This is useful to distinguish class methods and instance methods. Second, I use a hash prefix to identify singleton classes (`#MySingletonClass`).

Ruby has a flexible syntax, so few universal rules exist for things like indentation and the use of parentheses. Programmers tend to adopt the syntax that they find most readable in each specific case. In this book, I try to follow the most common conventions. For example, I skip parentheses when I call a method without parameters (as in `my_string.reverse`), but I tend to use parentheses when I pass parameters (as in `my_string.gsub("x", "y")`).

Some of the code in this book comes straight from existing open-source libraries. Some of these are standard Ruby libraries, so you should already have them. You can install the others with the `gem` command. For example, if I show you a piece of code from Builder 3.2.2, and you want to install the entire library to explore its source by yourself, then you can use `gem install builder -v 3.2.2`. Be aware of the version, because the code might have changed in more recent versions of Builder.

To avoid clutter (and make the code easier to understand in isolation), I'll sometimes take the liberty of editing the original code slightly. However, I'll do my best to keep the spirit of the original source intact.

Unit Tests

This book follows two developers as they go about their day-to-day work. As the story unfolds, you may notice that these two characters rarely write tests. Does this book condone untested code?